

FIAP — LINUX SERVICES APPLICATIONS
SEGURANÇA CIBERNÉTICA

Fedora Server 44

Instalação segura com LVM sobre LUKS, hardening de SSH e detecção de brute force

Grupo 1 — Documento de Pesquisa (Entregável 01)
Script do grupo: ssh-sentinel.sh — Detecção de brute force em SSH

Professor: Fábio Silva Pires de Oliveira Turma: 1TDCPG

Integrantes:

Bruno Tomé Duarte RM: 571712

Gabriel Zobolli Carnevalli RM: 571728

Heitor Fonseca Amorim dos Santos RM: 569118

Matheus Pascoal Craveiro RM: 572715

Entrega: 16 / 09 / 2026

Sumário

1. Introdução e contexto do trabalho
2. Fedora Server: posição no ecossistema Red Hat
3. Metodologia de pesquisa
4. Ambiente de laboratório e pré-requisitos
5. Esquema de particionamento de referência (LVM sobre LUKS)
6. Criptografia de disco com LUKS2
7. Gerenciamento de volumes com LVM
8. SELinux em modo enforcing
9. Hardening do serviço SSH
10. Script do grupo: ssh-sentinel.sh (detecção de brute force)
11. Limites de escopo e considerações legais
12. Conclusão
13. Referências

1. Introdução e contexto do trabalho

Este documento apresenta a pesquisa do Grupo 5, responsável pela distribuição Fedora Server, dentro do trabalho em grupo "Instalação segura de Linux, LVM sobre LUKS, serviço SSH e programação em Shell", da disciplina de Sistemas Operacionais Linux com foco em Cibersegurança. O objetivo do trabalho é sair do modelo de instalação automática típico de ambientes de estudo (o "next, next, finish") e reproduzir, em laboratório, as decisões de projeto que um administrador de sistemas toma antes de colocar um servidor Red Hat-family em produção: como particionar o disco, como criptografar os dados em repouso, como isolar cada partição contra os vetores de ataque mais comuns e como endurecer o único serviço que, por definição, fica exposto — o SSH.

O mercado corporativo — bancos, telecomunicações, governo e grande varejo — roda majoritariamente RHEL, Oracle Linux e seus derivados binariamente compatíveis (Rocky Linux, AlmaLinux, CentOS Stream). Quem aprende apenas em distribuições da família Debian encontra, ao migrar para esse ecossistema, um gerenciador de pacotes diferente (dnf/rpm em vez de apt/dpkg), um firewall diferente (firewalld em vez de ufw/iptables direto) e, principalmente, o SELinux — um controle de acesso obrigatório (MAC) que, no Debian, sequer vem habilitado por padrão. Este trabalho usa o Fedora Server como porta de entrada para essa família, já que ele é, ao mesmo tempo, gratuito, de uso livre e o ponto de origem de todo o código que eventualmente chega ao RHEL.

A pesquisa está organizada em três eixos que espelham a divisão de entregáveis do trabalho: (i) o recorte de pesquisa específico do grupo — a posição do Fedora no ecossistema Red Hat e seu ciclo de vida de aproximadamente 13 meses; (ii) os requisitos técnicos comuns aos seis grupos — esquema de particionamento com LVM sobre LUKS, SELinux enforcing e hardening de SSH; e (iii) o script de shell exclusivo do grupo — o ssh-sentinel.sh, voltado à detecção de tentativas de força bruta contra o SSH a partir do journald.

2. Fedora Server: posição no ecossistema Red Hat

2.1 Origem e mantenedor

O Fedora Project nasceu em novembro de 2003, quando a Red Hat decidiu dividir o antigo Red Hat Linux (RHL) em duas frentes: o **Red Hat Enterprise Linux (RHEL)**, produto comercial com suporte contratual, e o **Fedora**, uma distribuição comunitária de uso livre patrocinada pela Red Hat. A primeira versão, então chamada Fedora Core 1, saiu em 6 de novembro de 2003. Hoje o projeto é mantido por uma comunidade de milhares de colaboradores independentes, com a Red Hat (atualmente subsidiária da IBM) como patrocinadora principal — mas, segundo dados do próprio ecossistema, funcionários da Red Hat correspondem a uma minoria dos colaboradores ativos do projeto, a maioria não afiliada.

2.2 Fedora como upstream do RHEL

"Upstream" designa, em software livre, o repositório onde as contribuições e os lançamentos de um projeto realmente acontecem; o fluxo de código corre do upstream para os projetos "downstream" que dele dependem. No ecossistema Red Hat, essa cadeia tem hoje três elos: o Fedora Linux é o upstream mais próximo do usuário final e do desenvolvedor; o CentOS Stream funciona como um estágio intermediário de estabilização; e o RHEL é o produto downstream, comercialmente suportado, construído a partir dessa base.

Documentação oficial da Red Hat descreve exatamente esse encadeamento: o Fedora Project reúne código-fonte de dezenas de projetos upstream (kernel Linux, GNOME, systemd, Podman, utilitários GNU, servidores de display Wayland e X.org, entre outros) para montar o Fedora Linux; periodicamente, a Red Hat parte de um lançamento específico do Fedora para basear uma versão do RHEL. Em vez de compilar o kernel, o GNOME ou o systemd a partir do código "puro" (vanilla) desses projetos, a Red Hat parte das versões já integradas no Fedora — o que torna o Fedora um upstream do RHEL, com um upstream ainda mais distante formado pelos próprios projetos originais.

Esse desenho ficou mais explícito a partir da reorganização de 2020/2021, quando o CentOS Stream passou a ocupar formalmente a posição de "upstream de desenvolvimento" do RHEL, situado entre o Fedora e o produto final. O material da Fedora Magazine sobre a mudança reforça que o Fedora continua sendo o primeiro upstream do RHEL — é dali que toda versão do RHEL historicamente se origina, incluindo o RHEL 9. Na prática de laboratório, isso significa que recursos, ferramentas e políticas de segurança testados primeiro no Fedora (por exemplo, versões mais novas do SELinux, do firewalld ou do OpenSSH) tendem a aparecer, meses ou anos depois, de forma mais conservadora, em uma versão do RHEL.

Uma consequência prática dessa posição "na ponta" da cadeia é o ritmo de atualização de pacotes: o Fedora recebe compiladores, linguagens e bibliotecas em versões bem mais recentes que o RHEL, que prioriza estabilidade e compatibilidade binária de longo prazo — o RHEL disponibiliza versões mais novas de linguagens sob demanda por meio de Application Streams, sem trocar a base do sistema. Um padrão comum de mercado, inclusive, é desenvolver no Fedora e implantar em produção no RHEL.

2.3 Ciclo de lançamento e ciclo de manutenção (~13 meses)

É importante não confundir dois números diferentes quando se fala do Fedora: a cadência de lançamento e o tempo de suporte de cada versão. Segundo a documentação oficial do projeto, o Fedora Project lança uma nova versão principal do Fedora Linux aproximadamente a cada seis meses (tradicionalmente na quarta-feira de abril e de outubro), mas cada uma dessas versões recebe atualizações de manutenção por aproximadamente 13 meses.

O motivo dos 13 meses é aritmético: uma versão X do Fedora permanece recebendo pacotes e correções de segurança até cerca de quatro semanas depois do lançamento da versão X+2. Como o intervalo entre lançamentos é de seis meses, isso resulta em uma janela de suporte de aproximadamente treze meses por versão — suficiente para que um administrador, ao instalar a versão mais recente, tenha a opção de "pular" a versão seguinte e migrar diretamente para a próxima-mais-uma, sem nunca ficar com um sistema fora de suporte. Esse ciclo curto — bem mais curto que o de um RHEL, cujas versões maiores permanecem suportadas por anos — é o preço que o Fedora paga por estar na vanguarda tecnológica do ecossistema Red Hat, e é também o motivo pelo qual não é recomendado como distribuição de produção de longo prazo para quem não tem um processo maduro de atualização contínua.

Do ponto de vista de cibersegurança, esse ciclo tem uma implicação direta: um servidor Fedora fora de suporte deixa de receber correções para vulnerabilidades recém-descobertas no kernel, no OpenSSH, no SELinux e em qualquer outro componente do sistema base. Manter o inventário de versões e datas de fim de vida (EOL) é, portanto, parte do hardening — e não um detalhe apenas operacional.

2.4 Edições do Fedora e o recorte "Server"

O Fedora Project distribui diferentes "edições", cada uma otimizada para um público: Workstation (uso desktop), Server (infraestrutura de servidor), CoreOS (contêineres e workloads imutáveis), IoT, além de

variações de desktop como KDE Plasma. O Fedora Server é a edição relevante para este trabalho: instalação mínima, sem ambiente gráfico, orientada a rodar como servidor de rede — exatamente o perfil de instalação exigido pelo enunciado ("Server sem GUI / Minimal Install").

2.5 Versão utilizada pelo grupo

O grupo instalou especificamente o Fedora Server 44 (ISO Fedora-Server-dvd-x86_64-44-1.7.iso), a versão em ciclo de suporte ativo no período de execução do trabalho. Como qualquer versão do Fedora, ela está sujeita ao mesmo ciclo de manutenção de aproximadamente treze meses descrito na seção 2.3: passado esse prazo, a versão 44 deixa de receber atualizações de segurança, e a recomendação de mercado é migrar para uma versão mais recente antes do fim desse período.

3. Metodologia de pesquisa

A pesquisa combinou três tipos de fonte: (i) documentação oficial do Fedora Project (getfedora.org/wiki) sobre ciclo de lançamento e manutenção; (ii) documentação oficial da Red Hat (docs.redhat.com e redhat.com/blog) sobre a relação Fedora–CentOS Stream–RHEL e sobre os mecanismos de LUKS/LVM; e (iii) material técnico de terceiros (CIS Benchmarks, artigos técnicos e repositórios de referência no GitHub) para os parâmetros concretos de hardening de SSH e para o desenho do script de detecção de brute force. As afirmações de ciclo de vida do Fedora (seis meses de lançamento, treze meses de suporte) foram cruzadas em pelo menos três fontes independentes antes de serem tratadas como consenso. Trechos de comando e configuração foram validados contra a sintaxe atual das ferramentas (cryptsetup, lvm2, sshd, firewalld) e serão reexecutados no ambiente de laboratório do grupo antes da entrega final, conforme exigido pela seção de evidências do trabalho.

4. Ambiente de laboratório e pré-requisitos

O ambiente segue à risca os requisitos técnicos definidos para todos os seis grupos, reproduzidos abaixo com a adaptação ao Fedora Server:

Item	Configuração adotada pelo grupo
Hipervisor	VirtualBox / VMware / KVM-QEMU / Proxmox (a critério do grupo)
Firmware	UEFI habilitado — nunca BIOS legado
Disco principal	60 GB
Disco secundário	20 GB, adicionado somente após a instalação limpa
Memória / vCPU	4 GB / 2 vCPU
Rede	NAT ou Host-Only — nunca Bridge exposto à internet
Tipo de instalação	Fedora Server — instalação mínima, sem ambiente gráfico
SELinux	Enforcing do início ao fim da instalação e da configuração

Tabela 1 — Requisitos técnicos do laboratório, aplicados ao Fedora Server.

Duas precauções operacionais são obrigatórias antes de iniciar qualquer configuração: tirar um snapshot da VM logo após a instalação limpa e outro logo após a configuração do SSH (o ambiente vai quebrar em algum momento — é esperado); e anotar em local seguro a passphrase do LUKS, já que não existe procedimento

de recuperação sem ela e sem um backup do header — perder as duas coisas equivale a perder o dado em definitivo.

5. Esquema de particionamento de referência (LVM sobre LUKS)

O esquema de referência do trabalho não usa particionamento automático. Ele é desenhado antes da instalação, com um container LUKS2 único cobrindo praticamente todo o disco e, dentro dele, um volume group LVM fatiado em volumes lógicos dedicados a cada ponto de montagem sensível.

Partição / LV	Ponto de montagem	Tamanho	Observação
sda1	/boot/efi	1 GB (FAT32)	Fora da criptografia — exigido pelo firmware UEFI
sda2	/boot	1 GB (xfs)	Fora da criptografia — o GRUB precisa ler o kernel antes de existir chave
sda3	LUKS2 container	~58 GB	Volume Group vg_sistema sobre /dev/mapper/cryptlvm
lv_root	/	15 GB	
lv_var	/var	8 GB	
lv_varlog	/var/log	5 GB	Isolado do restante do /var
lv_vartmp	/var/tmp	3 GB	
lv_home	/home	10 GB	
lv_tmp	/tmp	3 GB	
lv_swap	swap	4 GB	
espaço livre	—	~9 GB	Reservado para snapshots de LVM

Tabela 2 — Esquema de particionamento de referência do trabalho.

5.1 Por que tudo dentro de um único container LUKS

A opção do esquema de referência é usar um único container criptografado hospedando o volume group inteiro, em vez de criptografar cada partição individualmente. A vantagem é operacional: uma única passphrase é solicitada no boot, e qualquer volume lógico novo criado depois — inclusive para acomodar o crescimento do disco secundário — já nasce criptografado, sem exigir uma nova camada de LUKS por LV.

5.2 Por que /boot fica fora da criptografia

O /boot/efi e o /boot permanecem fora do container LUKS porque o GRUB (o gerenciador de boot) precisa conseguir ler o kernel e a initramfs antes de qualquer chave de descriptografia estar disponível — é o próprio GRUB, ou um estágio anterior a ele, que solicita a passphrase para desbloquear o restante do disco. Esse desenho deixa um risco residual explícito: o conteúdo de /boot (kernel, initramfs, configuração do GRUB) fica exposto a quem tiver acesso físico ao disco, mesmo com o restante do sistema criptografado. Esse é exatamente o tipo de trade-off que o grupo precisa saber explicar na apresentação — a criptografia de disco integral (full-disk encryption) sem um /boot separado normalmente depende de mecanismos adicionais, como Secure Boot combinado com TPM2, fora do escopo mínimo deste trabalho.

5.3 Por que espaço livre no VG é requisito, não desperdício

Reservar cerca de 9 GB fora de qualquer volume lógico não é uma sobra de dimensionamento malfeito — é um requisito funcional do LVM. Sem espaço livre no volume group não é possível criar snapshots (usados, por exemplo, para tirar um backup consistente de um volume em uso) nem estender às pressas um volume lógico que encheu. Um VG a 100% de ocupação, portanto, não é uma otimização de uso de disco: é um problema operacional que o grupo deve evitar reproduzir.

5.4 Ciclo de vida do LVM: o disco secundário de 20 GB

Depois da instalação limpa, o trabalho exige anexar um segundo disco de 20 GB à VM e incorporá-lo ao volume group existente — o exercício de ciclo de vida do LVM que qualquer administrador Red Hat-family precisa dominar. Na prática, a sequência de comandos é:

1. `pvcreate /dev/sdb` — transforma o novo disco em um Physical Volume do LVM.
2. `vgextend vg_sistema /dev/sdb` — incorpora o novo PV ao volume group já existente.
3. `lvextend -L +<tamanho> /dev/vg_sistema/<lv_alvo>` — estende o volume lógico escolhido (por exemplo, `lv_var` ou `lv_home`, conforme a necessidade identificada em laboratório).
4. `xfs_growfs <ponto_de_montagem>` — expande o sistema de arquivos XFS a quente, sem desmontar, para ocupar o novo espaço do volume lógico.

Esse fluxo — crescer o armazenamento sem downtime — é justamente a vantagem operacional citada no enunciado do trabalho como motivo para não usar particionamento automático em ambiente de produção: com discos físicos fixos e sem LVM, o mesmo crescimento normalmente exigiria reinstalação ou uma reorganização manual arriscada das partições.

6. Criptografia de disco com LUKS2

LUKS (Linux Unified Key Setup) é o formato padrão de criptografia de disco em nível de bloco em distribuições Red Hat-family, incluindo o Fedora. Segundo a documentação oficial da Red Hat, o LUKS usa a infraestrutura já existente do device mapper do kernel, oferece fortalecimento de passphrase para dificultar ataques de dicionário, e permite múltiplos slots de chave — ou seja, mais de uma senha ou chave pode desbloquear o mesmo volume, útil para recuperação e para rotação de credenciais sem reencriptar tudo.

A partir do RHEL/Fedora atuais, o formato padrão é o LUKS2, sucessor do LUKS1. As vantagens documentadas do LUKS2 incluem: metadados em formato de texto JSON (mais fácil de auditar e de recuperar em caso de corrupção), redundância de metadados com reparo automático a partir de uma cópia, até 32 slots de chave (contra 8 do LUKS1) e suporte a reencrytação online — ou seja, é possível trocar a chave de volume ou o algoritmo de criptografia de um dispositivo já em uso, sem precisar desmontar o sistema de arquivos, algo que o LUKS1 não permite.

É importante frisar uma limitação conceitual do LUKS destacada pela própria documentação da Red Hat: ele protege os dados apenas enquanto o sistema está desligado. Uma vez que a máquina está ligada e o LUKS já desbloqueou o disco, os arquivos ficam disponíveis normalmente para quem tiver acesso ao sistema em execução — o LUKS não substitui controle de acesso em nível de sistema operacional (que é papel do SELinux, das permissões Unix e do próprio SSH endurecido), nem criptografia em nível de arquivo para cenários que exijam chaves distintas por usuário.

7. Gerenciamento de volumes com LVM

O LVM (Logical Volume Manager) introduz uma camada de abstração entre os discos físicos e os sistemas de arquivos: um ou mais discos físicos (Physical Volumes) são agrupados em um Volume Group, de onde são fatiados os Volumes Lógicos (Logical Volumes) que efetivamente recebem um sistema de arquivos e um ponto de montagem. No esquema deste trabalho, o Physical Volume do LVM não é um disco bruto, e sim o mapeamento aberto pelo LUKS (/dev/mapper/cryptlvm) — por isso "LVM sobre LUKS": a criptografia fica na camada mais baixa, e a flexibilidade de redimensionamento do LVM opera por cima dela, de forma transparente.

As vantagens de separar / em múltiplos volumes lógicos, em vez de uma única partição raiz, são tanto operacionais quanto de segurança: cada LV pode crescer independentemente (como demonstrado no fluxo de extensão do disco secundário), pode ter seu próprio sistema de arquivos e — o ponto central da próxima seção — pode receber opções de montagem restritivas específicas para o tipo de dado que ali é guardado, sem afetar o restante do sistema.

8. SELinux em modo enforcing

O SELinux (Security-Enhanced Linux) é um módulo de controle de acesso obrigatório (Mandatory Access Control) integrado ao kernel Linux, desenvolvido originalmente pela NSA em parceria com a comunidade e mantido como parte central da estratégia de segurança das distribuições Red Hat-family. Diferentemente do controle de acesso discricionário tradicional do Unix (permissões de dono/grupo/outros), o SELinux aplica políticas obrigatórias baseadas em rótulos (contexts) atribuídos a cada processo e a cada arquivo — mesmo um processo rodando como root fica restrito ao que sua política permite.

O trabalho exige SELinux em modo enforcing do início ao fim da instalação e da configuração — nunca permissive e nunca disabled. Essa exigência não é apenas formal: entre os descontos previstos na rubrica, desligar o SELinux custa dez pontos, o maior desconto pontual do trabalho, refletindo o quanto esse controle é considerado central na filosofia de segurança Red Hat. Comandos de verificação relevantes incluem getenforce (mostra o modo atual) e sestatus (mostra o status detalhado da política); ambos fazem parte das evidências obrigatórias da seção de Sistema e SELinux.

Uma implicação prática do SELinux enforcing, relevante tanto para o particionamento quanto para o SSH, é que mudanças que fogem do padrão — como mover o SSH para uma porta não padrão — exigem ajustar o rótulo (contexto) do SELinux correspondente com a ferramenta semanage, sob risco de o serviço falhar silenciosamente ou de o SELinux bloquear a nova porta mesmo com o firewall liberando o tráfego.

9. Hardening do serviço SSH

O OpenSSH, assim como a maioria dos serviços, sai da instalação padrão em um estado permissivo: aceita autenticação por senha e, historicamente, permite login direto como root. Tirá-lo desse padrão é trabalho manual de configuração — exatamente o tipo de tarefa que se cobra de um administrador de sistemas, e o eixo central deste trabalho junto ao particionamento seguro.

9.1 Controles exigidos pelo trabalho

Controle	Objetivo de segurança
Par de chaves ed25519; PasswordAuthentication no	Elimina o vetor de força bruta contra senha — só quem possui a chave privada autentica

Controle	Objetivo de segurança
PermitRootLogin no	Acesso sempre nominal por usuário identificável; privilégio elevado só via sudo, com trilha de auditoria
Porta não padrão + rótulo SELinux ajustado (semanage)	Reduz o ruído de varreduras automatizadas contra a porta 22; exige ajuste explícito de política
AllowGroups com grupo dedicado	Lista de permissão explícita — só contas do grupo autorizado alcançam o SSH
MaxAuthTries 3 / LoginGraceTime 30 / ClientAliveInterval 300	Limita tentativas por conexão, tempo de autenticação e sessões ociosas
update-crypto-policies; cifras e MACs legados removidos	Remove algoritmos criptográficos fracos/obsoletos do conjunto negociável
firewalld liberando só a porta nova, com rate limit	Reduz superfície de rede exposta e limita a taxa de novas conexões
Banner de aviso legal em /etc/issue.net	Formaliza o aviso de uso autorizado antes da autenticação
Registro de autenticação no journald	Base de evidência para auditoria e para o script de detecção de brute force

Tabela 3 — Controles de hardening do SSH exigidos pelo trabalho, com o objetivo de segurança de cada um.

Sobre os parâmetros numéricos: benchmarks de hardening amplamente usados como referência de mercado (CIS Benchmark para RHEL/CentOS) recomendam MaxAuthTries igual a 4 ou menos, e módulos de automação de hardening consolidados (como o dev-sec/ansible-collection-hardening) adotam LoginGraceTime na casa de 30 segundos como prática comum — valores compatíveis com os solicitados no trabalho (MaxAuthTries 3, LoginGraceTime 30), na extremidade mais restritiva dessa faixa.

9.2 A regra de ouro do hardening remoto

O procedimento operacional mais importante desta seção não é técnico, é de disciplina: nunca encerrar a sessão SSH atual antes de validar, em uma segunda sessão aberta em paralelo, que a nova configuração realmente funciona. Um sshd_config mal formado, uma regra de firewall equivocada ou um contexto de SELinux desalinhado podem derrubar o acesso remoto — e, em uma VM de laboratório, o custo do erro é apenas voltar a um snapshot; em produção, o mesmo erro vira um chamado de emergência fora do horário comercial.

10. Script do grupo: ssh-sentinel.sh (detecção de brute force)

O script exclusivo do Grupo 5 é o ssh-sentinel.sh, cuja responsabilidade é ler o journald, agregar falhas de autenticação por endereço IP, aplicar um limiar configurável e bloquear os endereços que ultrapassam esse limiar no firewalld, com um TTL (tempo de vida) para o bloqueio. O script mantém obrigatoriamente uma lista de exceções (allowlist) para não bloquear redes de gerência ou o próprio time, e produz um relatório com os dez IPs e os dez usuários mais visados.

10.1 Arquitetura proposta

- Coleta: leitura do journald filtrada pela unit do sshd (journalctl -u sshd -o json ou equivalente), restrita à janela de tempo configurada (findtime).

6. Parsing: extração do endereço IP de origem e do nome de usuário tentado a partir das mensagens de falha de autenticação ("Failed password", "Invalid user", "authentication failure"), evitando expressões regulares frágeis que quebrem com pequenas variações de locale ou de versão do OpenSSH.
7. Agregação: contagem de falhas por IP dentro da janela de tempo; separação entre falhas por senha inválida, usuário inexistente e falha de chave pública, já que representam sinais de risco distintos.
8. Decisão: comparação da contagem contra o limiar configurável (maxretry); IPs presentes na allowlist nunca são bloqueados, independentemente da contagem.
9. Ação: inserção de uma regra temporária no firewalld (rich rule com reject/drop) para o IP ofensor, com TTL definido; remoção automática do bloqueio ao expirar o TTL, seja por temporizador do systemd, seja por verificação na execução seguinte do script.
10. Relatório: saída em texto (preferível a captura de tela, por ser auditável) com o top 10 de IPs e o top 10 de usuários visados na janela analisada, e registro em log próprio em /var/log/, conforme exigido pelos requisitos comuns aos seis grupos.

10.2 Requisitos comuns aplicados a este script

Como todos os seis scripts do trabalho, o ssh-sentinel.sh precisa seguir modo estrito (set -euo pipefail), ter no mínimo cinco funções (sem lógica solta no corpo do script), aceitar parâmetros via getopts ou case (com -h/--help obrigatórios), verificar privilégio via EUID e dependências via command -v, registrar logs com timestamp e nível INFO/WARN/ERROR, usar os códigos de saída padronizados (0 sucesso, 1 achado, 2 erro de uso, 3 dependência ausente), ser idempotente, não conter segredos no código, limpar temporários com trap e mktemp, passar no shellcheck sem erros, ser reversível (quem bloqueia um IP precisa saber desbloquear) e ter comentários e mensagens em português.

Um cuidado de projeto específico deste script — análogo ao exigido explicitamente do script de firewall do Grupo 4 — é nunca bloquear o próprio administrador: a allowlist deve, no mínimo, cobrir a rede de gerência da VM e o IP a partir do qual o grupo está testando o script, sob pena de o próprio ssh-sentinel.sh causar a negação de serviço que deveria prevenir.

10.3 Comparação conceitual com o fail2ban

O enunciado do trabalho pede explicitamente a comparação do script com o fail2ban, ferramenta de referência de mercado para esse tipo de proteção. Segundo documentação técnica consolidada, o fail2ban opera com o mesmo princípio geral do ssh-sentinel.sh — monitoramento de log seguido de banimento automático no firewall — mas como um daemon persistente: ele acompanha os logs continuamente (por padrão via inotify, com detecção quase em tempo real), usa filtros baseados em expressões regulares específicos por serviço ("jails"), e assim que um IP ultrapassa o número de tentativas permitido (maxretry) dentro da janela configurada (findtime), é banido pelo tempo definido em bantime. O fail2ban também oferece um jail especial, o recidive, que aplica banimentos progressivamente mais longos a reincidentes.

Aspecto	ssh-sentinel.sh (script do grupo)	fail2ban (referência de mercado)
Modelo de execução	Execução sob demanda ou agendada (cron/timer systemd), lote sobre a janela configurada	Daemon persistente, com leitura de log em tempo quase real via inotify

Aspecto	ssh-sentinel.sh (script do grupo)	fail2ban (referência de mercado)
Fonte de eventos	journald (journalctl -u sshd)	Arquivo de log tradicional (/var/log/auth.log ou /var/log/secure) ou journald, conforme backend
Ação de bloqueio	Rich rule do firewalld com TTL definido no próprio script	Integração com iptables/nftables/firewalld/ufw via módulo de ação (banaction)
Escopo de filtros	Um único serviço (SSH), com regras de parsing definidas pelo grupo	Múltiplos jails prontos para dezenas de serviços (SSH, HTTP, mail, etc.)
Reincidência	Não nativo — poderia ser adicionado como extensão futura	Jail dedicado (recidive) para bans progressivos
Maturidade/comunidade	Projeto acadêmico, código auditável e simples de entender integralmente	Ferramenta madura, testada em produção há anos, com grande base de filtros

Tabela 4 — Comparação conceitual entre o script do grupo e o fail2ban.

A conclusão pedagógica dessa comparação é que o ssh-sentinel.sh não pretende substituir o fail2ban em um ambiente de produção real — pretende reproduzir, de forma simplificada e totalmente auditável pelo próprio grupo, o mesmo raciocínio de detecção que sustenta uma ferramenta madura como o fail2ban, deixando explícitas decisões de projeto (janela de tempo, limiar, allowlist, TTL) que, em uma ferramenta pronta, ficam escondidas atrás de um arquivo de configuração.

11. Limites de escopo e considerações legais

Todo o trabalho — incluindo a simulação de força bruta usada para validar o ssh-sentinel.sh — é executado exclusivamente em máquina virtual do próprio grupo. Nenhum procedimento deste trabalho (varredura de portas, tentativa de autenticação, alteração de firewall, execução de script) pode ser aplicado contra equipamento da instituição, de empresa, de colega ou de terceiros. A simulação de brute force do script do Grupo 5 deve partir de uma segunda VM, em rede isolada, e ter como alvo exclusivamente o próprio servidor do grupo.

Essa exigência de escopo não é apenas uma boa prática acadêmica: acesso não autorizado a sistema informatizado é crime tipificado no Artigo 154-A do Código Penal brasileiro, incluído pela Lei nº 12.737/2012. Qualquer evidência de atuação fora do escopo definido resulta em nota zero para o grupo e encaminhamento à coordenação do curso, conforme deixado explícito pelo enunciado do trabalho.

O uso de inteligência artificial generativa como apoio à pesquisa, à redação e à revisão deste documento é permitido, desde que declarado no arquivo USO-DE-IA.md do repositório do grupo — declaração que será incluída junto à entrega final. Conteúdo tecnicamente incorreto continua sendo penalizado independentemente de ter tido apoio de IA generativa em sua produção.

12. Conclusão

A pesquisa deste grupo mostra que o Fedora Server ocupa uma posição estrutural no ecossistema Red Hat: é o upstream mais próximo do desenvolvedor e do usuário final, o primeiro lugar onde novos recursos de segurança e novas versões de componentes (kernel, SELinux, OpenSSH, firewalld) são testados antes de, meses ou anos depois, chegarem de forma mais conservadora ao RHEL via CentOS Stream. Seu ciclo de

aproximadamente treze meses de suporte por versão — decorrente de lançamentos a cada seis meses com uma janela de sobreposição de quatro semanas — exige do administrador uma disciplina de atualização que distribuições de ciclo mais longo dispensam.

No plano técnico, o trabalho evidenciou que segurança, neste ecossistema, é essencialmente uma questão de configuração deliberada, não de instalação padrão: o esquema de LVM sobre LUKS2 protege os dados em repouso e mantém a flexibilidade de crescimento sem downtime; as opções de montagem restritivas (nodev, nosuid, noexec) fecham o vetor de escalonamento de privilégio mais clássico via diretórios graváveis; o SELinux enforcing aplica controle de acesso obrigatório mesmo sobre processos privilegiados; e o hardening do SSH — autenticação só por chave, sem root direto, com lista de permissão explícita e limites de tentativa — reduz drasticamente a superfície de ataque do único serviço que, por definição, precisa ficar acessível de fora. O script `ssh-sentinel.sh` fecha esse ciclo transformando os registros de autenticação do `journald` em uma resposta automática e auditável contra tentativas de força bruta, com o cuidado de nunca bloquear o próprio administrador.

13. Referências

FEDORA PROJECT. Fedora Linux Release Life Cycle. Fedora Docs / [getfedora.org](https://getfedora.org/wiki/Releases/Schedule), 2026. Disponível em: <https://getfedora.org/wiki/Releases/Schedule>. Acesso em: 15 set. 2026. [fonte primária]

FEDORA PROJECT. Fedora Linux Releases. [getfedora.org](https://getfedora.org/wiki/Releases), 2026. Disponível em: <https://getfedora.org/wiki/Releases>. Acesso em: 15 set. 2026. [fonte primária]

RED HAT, INC. What is an open source upstream? Red Hat Blog, 2026. Disponível em: <https://www.redhat.com/en/blog/what-open-source-upstream>. Acesso em: 15 set. 2026. [fonte primária]

RED HAT, INC. What's the difference between Fedora and Red Hat Enterprise Linux? Red Hat Topics, [s.d.]. Disponível em: <https://www.redhat.com/en/topics/linux/fedora-vs-red-hat-enterprise-linux>. Acesso em: 15 set. 2026. [fonte primária]

RED HAT, INC. Chapter 9/20 — Encrypting block devices using LUKS. Red Hat Enterprise Linux 9 Security Hardening / Managing Storage Devices, docs.redhat.com, 2026. Disponível em: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/security_hardening/encrypting-block-devices-using-luks_security-hardening. Acesso em: 15 set. 2026. [fonte primária]

RED HAT, INC. LUKS versions in RHEL. docs.redhat.com, 2026. Disponível em: https://docs.redhat.com/ko/documentation/Red_Hat_Enterprise_Linux/10/html/managing_storage_devices/luks-versions-in-rhel. Acesso em: 15 set. 2026. [fonte primária]

CENTER FOR INTERNET SECURITY (CIS). CIS Red Hat Enterprise Linux 9 Benchmark — SSH Server Configuration (seção 5.2, referenciada via ato-pathways.com e [finalduty/cis_benchmarks_audit](https://github.com/finalduty/cis_benchmarks_audit)). 2025-2026. Disponível em: <https://ato-pathways.com/catalogs/xccdf/profiles/60/items> e https://github.com/finalduty/cis_benchmarks_audit. Acesso em: 15 set. 2026.

FEDORA MAGAZINE. Fedora and CentOS Stream. fedoramagazine.org, 2019 (contexto histórico da relação Fedora–CentOS Stream–RHEL, ainda válido conceitualmente). Disponível em: <https://fedoramagazine.org/fedora-and-centos-stream/>. Acesso em: 15 set. 2026.

WIKIPEDIA. Red Hat Enterprise Linux derivatives / Fedora Project (verbetes de consulta geral para contexto histórico e de fundação). Disponível em: https://en.wikipedia.org/wiki/Red_Hat_Enterprise_Linux_derivatives. Acesso em: 15 set. 2026.

LANSWEEPER. Fedora Linux End of Life. lansweeper.com, 2025-2026 (tabela de versões e datas de EOL, cruzada com a documentação oficial). Disponível em: <https://www.lansweeper.com/blog/eol/fedora-linux-end-of-life/>. Acesso em: 15 set. 2026.

STACKHARBOR. Fail2ban for SSH — banning brute-force without locking yourself out. stackharbor.com, 2026. Disponível em: <https://stackharbor.com/en/knowledge-base/fail2ban-ssh-protection/>. Acesso em: 15 set. 2026.

GITHUB — dev-sec/ansible-collection-hardening; Krishcalin/Linux-Hardening. Repositórios de referência para parâmetros de hardening de SSH e CIS Benchmark aplicados via automação. Disponível em: <https://github.com/dev-sec/ansible-collection-hardening> e <https://github.com/Krishcalin/Linux-Hardening>. Acesso em: 15 set. 2026.